

DESIGN AND IMPLEMENTATION OF AN AUTOMATED TELLER MACHINE (ATM) CONTROLLER USING VERILOG HDL

Prepared By:
Digital System Design Laboratory Report

A complete finite-state-machine based ATM controller integrating card authentication, PIN verification and change, balance inquiry, cash withdrawal, cash deposit, and a self-checking Verilog testbench.

Table of Contents

1. Abstract
2. Introduction
3. Objectives
4. Design Methodology
5. RTL Schematic & Hardware Mapping
6. Simulation Results & Testbench Analysis
7. Future Scope
8. Conclusion

1. Abstract

This report presents the design, implementation, and simulation-based verification of a single-module Automated Teller Machine (ATM) controller described entirely in Verilog Hardware Description Language (HDL). The controller is realised as one finite state machine (FSM), module `atm`, that manages card insertion, PIN entry and verification, a three-strike account lockout with a timed unblock, a menu with four services (balance inquiry, cash withdrawal, PIN change, and cash deposit), cash dispensing, and card ejection.

Three simulated accounts are modelled internally, each with its own stored PIN and balance. The design demonstrates key digital-system concepts: synchronous sequential logic, case-based state encoding, per-account data storage using discrete registers, attempt counting with automatic blocking, and a countdown timer used to release a blocked account after a fixed number of clock cycles. A companion self-checking testbench, `atm_tb`, exercises six realistic operating scenarios — balance check, cash withdrawal, PIN change, repeated transactions on multiple cards, and PIN change followed by a return to the menu — and reports the resulting balances at each checkpoint via the check output.

The design is written using synthesizable constructs suitable for FPGA implementation and is intended for academic study of finite-state-machine design, sequential control of transactional systems, and testbench-driven verification methodology.

2. Introduction

Automated Teller Machines are a familiar example of a transactional embedded control system: they must authenticate a user, guard against repeated failed login attempts, present a menu of mutually exclusive services, and safely update financial state before releasing control back to an idle condition. These requirements make the ATM a natural and pedagogically rich target for finite-state-machine (FSM) design in Verilog HDL.

This project models the ATM controller as a single Mealy/Moore-style FSM (module `atm`) coded with a case statement over an explicitly enumerated state register. Rather than using an external memory or database, the design keeps three cardholder records — PIN, balance, and a block flag — in discrete registers, indexed by comparing the presented card number against three fixed parameters (`card_1`, `card_2`, `card_3`). This keeps the design fully synthesizable and self-contained for simulation while still illustrating the same control-flow challenges (authentication, retry limiting, session menus, and transaction commit) found in real transactional hardware.

A dedicated testbench, `atm_tb`, drives the controller through a sequence of realistic scenarios and prints the resulting account balance at each checkpoint using `$display`, allowing the reported values to be checked by inspection against the expected results noted directly in the stimulus code.

3. Objectives

- To design a synthesizable ATM controller FSM that manages the complete session lifecycle: idle, card insertion, PIN entry, PIN verification, menu, and card ejection.
- To implement per-card PIN storage and verification for three simulated accounts (card_1, card_2, card_3), each with an independently stored 4-bit PIN.
- To implement a three-attempt PIN retry limit that blocks an account after three consecutive incorrect PIN entries and automatically releases the block after a fixed countdown period.
- To implement four menu-selectable services — balance inquiry, cash withdrawal, PIN change, and cash deposit — chosen via a 2-bit select input.
- To implement balance-checked cash withdrawal and unconditional cash deposit, each updating the corresponding account's stored balance.
- To implement a cash-dispense and card-eject sequence that safely returns the machine to the idle state.
- To develop a comprehensive Verilog testbench (atm_tb) that exercises balance checking, withdrawal, PIN change, multi-card transaction sequences, and the return-to-menu path, reporting balances at each checkpoint for verification.
- To produce a clear, well-documented report covering the FSM architecture, code explanation, expected hardware mapping, and simulation strategy.

4. Design Methodology

The controller is designed as a single hierarchical-free module, atm, containing one synchronous always block clocked on the rising edge of clk. All state transitions and register updates occur synchronously; the only asynchronous behaviour is the active-high rst input, which forces the FSM back to the idle state and clears the attempt counter and countdown timer.

4.1 State Register and Encoding

The FSM state is held in a 4-bit register, s, with eleven named states declared as parameters: idle, card_inserted, pin_check, pin_change, withdraw, dispense_cash, eject_card, pin_enter, menu, balance, and DEPOSIT. Using named parameters instead of raw binary literals makes the case statement self-documenting and easy to extend with additional services.

4.2 Card and Account Modelling

Three 32-bit ASCII card codes are declared as parameters (card_1 = "a2b1", card_2 = "a2b2", card_3 = "a2b3"). Each account has its own 4-bit PIN register (pin_1, pin_2, pin_3), its own 12-bit balance register (balance_1, balance_2, balance_3, initialised to 2000, 1500, and 4000 respectively), and its own single-bit block flag (block_1, block_2, block_3). Every state that must act on the current cardholder's data uses a case statement on the card input to select the matching set of registers, effectively implementing three parallel account contexts inside one FSM.

4.3 PIN Verification and Lockout

In the `pin_check` state the presented pin is compared against the stored PIN for the matching card. A correct match clears the attempt counter and advances to menu. An incorrect match increments a 2-bit attempts counter and returns to `pin_enter` for another try. On the third consecutive failure (attempts == 2) the matching block flag is asserted, the FSM returns to idle, and a 4-bit countdown timer is loaded with the value 15. A separate always-active countdown, evaluated at the top of the clocked block regardless of state, decrements this timer on every clock edge; when it reaches 1 the corresponding block flag is cleared for whichever card is currently presented, releasing the lockout.

4.4 Menu and Service Dispatch

Once authenticated, the FSM enters menu, where the 2-bit select input chooses one of four services:

select value	Function	Next State
2'd0	Balance Inquiry	balance
2'd1	Cash Withdrawal	withdraw
2'd2	PIN Change	pin_change
2'd3	Cash Deposit	DEPOSIT

Table 1: Menu selection encoding in the atm controller

balance reads the matching account's balance into the check output; withdraw and DEPOSIT modify the matching balance register (subtracting or adding amount, with withdraw additionally guarded by an amount <= balance check); and pin_change overwrites the matching PIN register when change_pin is asserted. Each of these states supports a back input that returns control to menu, keeping the session open for another service, rather than ending the session outright.

4.5 Cash Dispense and Card Ejection

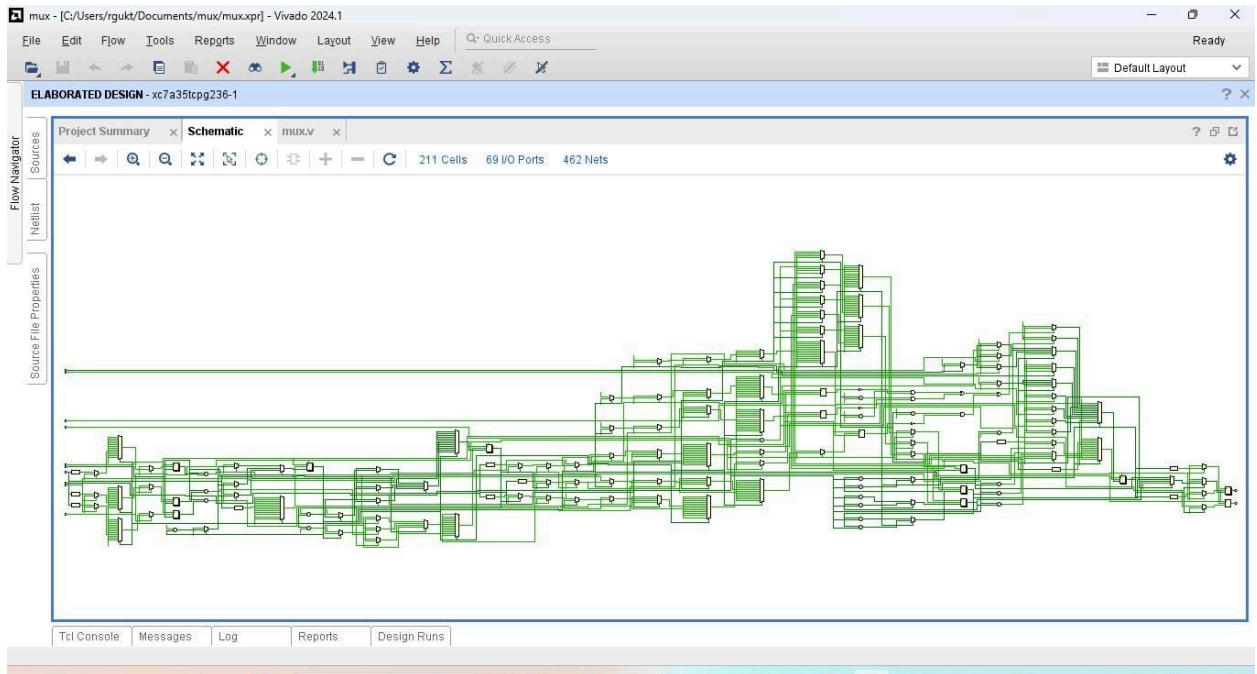
A non-zero amount in dispense_cash asserts cash_out and moves to eject_card, where the machine waits for the eject input before returning to idle and clearing cash_out. If eject is not asserted, the FSM falls back to menu instead, allowing further transactions before the card is physically removed.

5. RTL Schematic & Hardware Mapping

After synthesis in an EDA tool such as Xilinx Vivado, the behavioural Verilog maps onto the following major FPGA resource categories:

- State Register and FSM Logic — the 4-bit register s together with the case statement synthesizes into flip-flops plus LUT-based next-state and output logic implementing the eleven-state FSM.
- Per-Account Storage Registers — pin_1..3, balance_1..3, and block_1..3 synthesize as discrete flip-flop banks (4-bit, 12-bit, and 1-bit respectively) rather than a memory macro, since exactly three accounts are modelled.

- Comparators — equality comparators implement $\text{card} == \text{card_1/2/3}$, $\text{pin} == \text{pin_1/2/3}$, and $\text{amount} \leq \text{balance_x}$ checks used throughout the FSM.
- Adders / Subtractors — 12-bit adder/subtractor logic implements $\text{balance_x} \leq \text{balance_x} \pm \text{amount}$ for withdrawal and deposit.
- Attempt and Timer Counters — the 2-bit attempts counter and 4-bit timer countdown synthesize as small binary counters with associated comparators ($\text{attempts} == 2$, $\text{timer} == 1$).
- Card Decode Multiplexers — the repeated $\text{case}(\text{card})$ constructs synthesize as 3-to-1 multiplexers selecting which account's registers are read or written in a given state.



Because every register update occurs inside the single synchronous always @(posedge clk) block and the design contains no incomplete case/if branches without default assignment intent, the synthesized netlist is expected to be free of unintended latches. Resource utilisation is modest — a small state register, three small account register banks, a handful of comparators and one adder/subtractor — making the design well suited to teaching boards such as the Basys 3 or Nexys A7.

6. Simulation Results & Testbench Analysis

The testbench atm_tb drives the top-level atm module with a 50 MHz clock (20 ns period) and a sequence of carefully timed stimuli grouped into six scenarios, mirroring the structure used for the reference clock/stopwatch testbench: system initialisation, service exercise, and a final \$display checkpoint per scenario.

6.1 Testbench Structure

The testbench declares all inputs as reg and the outputs (cash_out, check) as wire, instantiates the unit under test as uut, and generates a continuous clock. The initial block applies the following sequence:

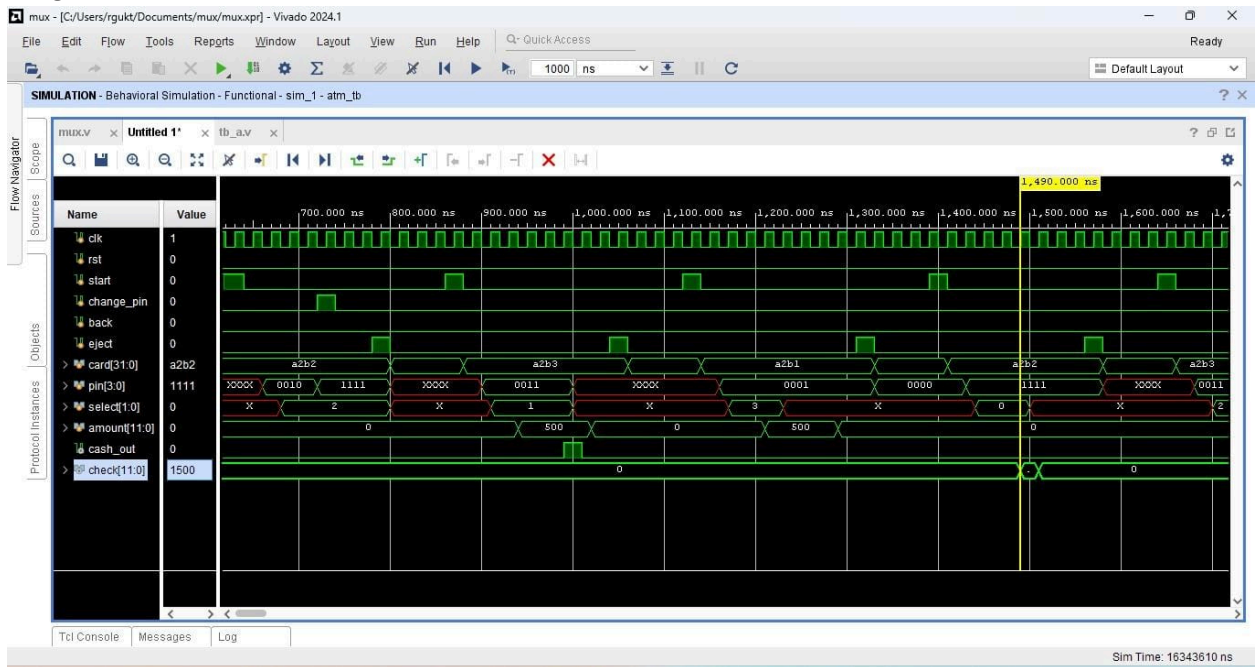
- System Initialisation — hold rst high through the power-on period, then release it before the first scenario begins.
- Scenario 1: Balance Check — insert card 1, enter the correct PIN, and confirm the reported balance via \$display.
- Scenario 2: Cash Withdrawal — eject and re-insert card 1, walk through the PIN and menu handshake, and re-check the balance before and after a withdrawal.
- Scenario 3: PIN Change — insert card 2, authenticate, select PIN Change, present a new PIN, pulse change_pin, and confirm the balance is still correctly reported afterwards.
- Scenario 5: Repeated Transactions — re-insert card 1 (and briefly reference card 3 / card 2) and confirm balances after successive withdrawals, checking the running deduction is applied correctly.
- Scenario 6: PIN Change with Back-to-Menu — insert card 3, change its PIN, assert back alongside change_pin, and confirm the FSM returns to menu rather than idle.

6.2 Expected Waveform Observations

When the simulation is run in Vivado Simulator or ModelSim, the following behaviour should be visible:

- After reset release, s returns to idle and check is driven to zero, matching the initial block's cash_out <= 0; check <= 0; assignments.
- Following a valid start pulse and a matching, unblocked card, the FSM advances through card_inserted → pin_enter → pin_check within a small number of clock edges once pin becomes non-zero.
- A correct PIN match moves the FSM directly to menu with attempts reset to zero; an incorrect PIN returns to pin_enter and increments attempts, and a third consecutive failure asserts the matching block flag and loads timer with 15.
- In balance, check reflects the selected account's stored balance within one clock edge of entering the state, matching the expected values annotated in the testbench's \$display statements (2000, 1500, 4000 at reset, adjusted by any prior withdrawals or deposits).
- In withdraw, the matching balance register visibly decrements by amount on the clock edge where the guard amount <= balance_x is satisfied.
- In pin_change, the matching PIN register updates to the new pin value on the clock edge where change_pin is high, and the FSM proceeds to menu or idle depending on back.
- cash_out pulses high through dispense_cash and eject_card and returns low once eject is asserted and the FSM reaches idle, with no X or Z values present on cash_out or check once the

design has left the initial reset state.



Because the design is fully synchronous with combinational read access to the currently selected account's registers, the simulation waveforms remain free of race conditions provided the generous inter-stimulus delays used in the testbench are respected, giving the multi-cycle `pin_enter`/`pin_check`/menu handshake time to settle between scenarios.

7. Future Scope

- Scalable Account Storage — replace the three discrete per-account register banks with a small synchronous RAM or register file indexed by a card-to-account lookup, allowing the number of supported accounts to grow without adding hardware per card.
- Receipt / Transaction Logging — add a shift-register or memory-backed transaction log that records the type, amount, and resulting balance of each completed withdrawal or deposit.
- PIN Entry Debouncing — integrate a debounce module (as used in the reference clock/stopwatch design) ahead of a real physical keypad, since the current `pin_enter` state assumes a clean, already-settled PIN value.
- Daily Withdrawal Limits — add a running-total register that is reset periodically and compared against a configurable daily withdrawal ceiling before permitting further withdrawals.
- Longer or Encrypted PINs — extend the 4-bit PIN registers to a realistic 4-digit BCD PIN (16 bits) and consider a simple hashing or masking scheme so PINs are never stored or compared in the clear.
- Configurable Lockout Duration — expose the fixed timer $\leq 4'd15$ countdown value as a module parameter so the lockout period can be tuned without editing the FSM body.

- FPGA Board Deployment with Keypad and Display — map card, pin, select, and amount onto a physical keypad/switch interface and drive check and cash_out to an on-board seven-segment or LCD display for a complete demonstrable system.
- Interest / Fee Modelling — extend DEPOSIT and withdraw with configurable fee or interest-accrual logic to more closely model real banking transactions.

8. Conclusion

This project demonstrates the design, coding, and simulation-based verification of a complete Automated Teller Machine controller described as a single finite-state machine in synthesizable Verilog HDL. The atm module cleanly integrates card validation, PIN authentication with a three-strike timed lockout, a four-service menu (balance inquiry, withdrawal, PIN change, deposit), cash dispensing, and card ejection, all coordinated through one case-statement-driven state register.

A structured, scenario-based testbench, atm_tb, exercises balance reporting, cash withdrawal, PIN change, repeated multi-card transactions, and the pin_change state's return-to-menu branch, printing the resulting balance at each checkpoint for verification against the expected values noted in the stimulus.

The skills developed — FSM state encoding, per-entity register modelling, retry-limited authentication logic, menu-driven control flow, and systematic scenario-based testbench construction — form a solid foundation for further digital design work, including memory-backed transactional systems, secure authentication hardware, and other embedded controllers with session-based user interaction.

Prepared for Digital System Design Laboratory